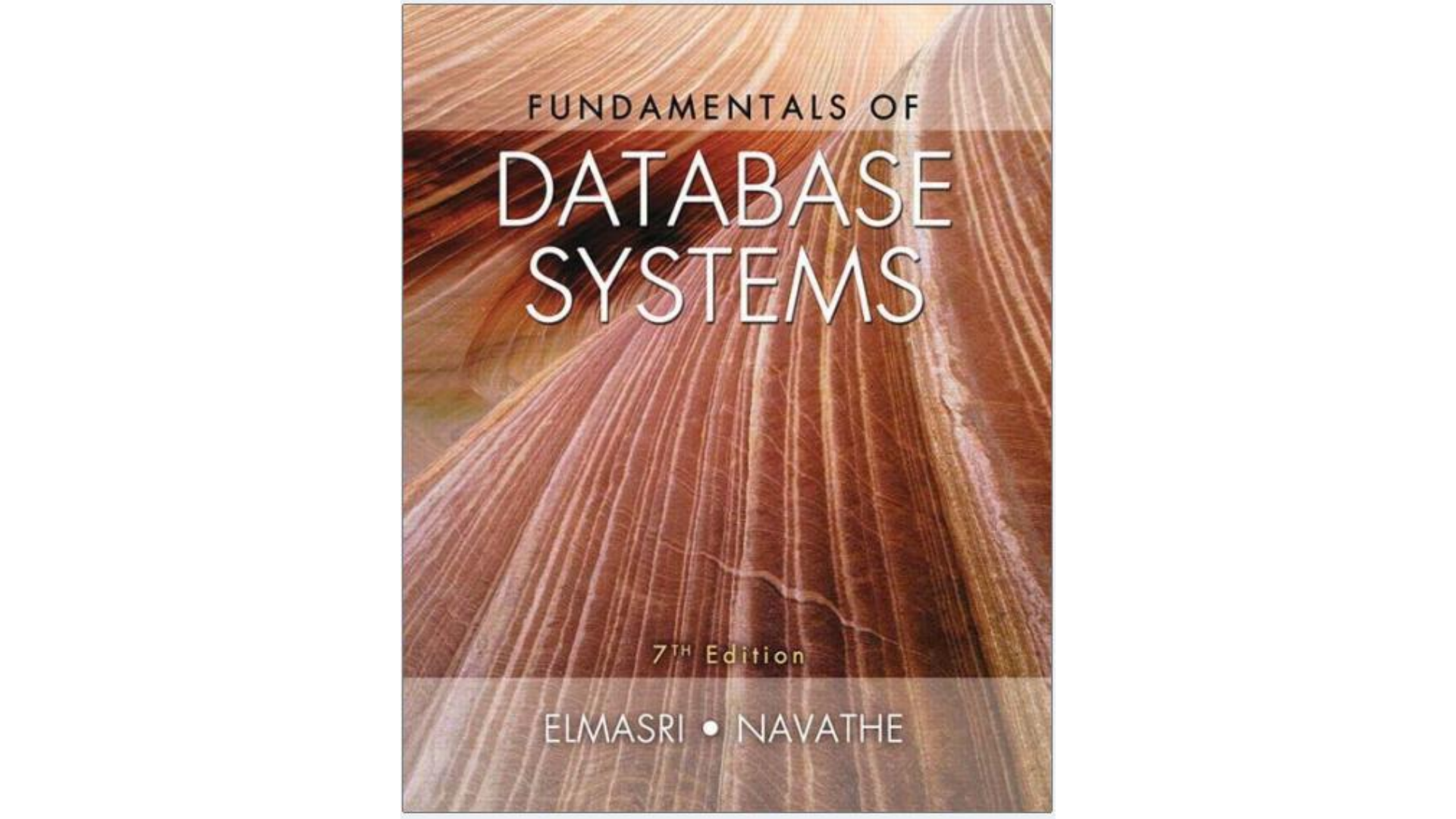


TD7: Ingeniería de Datos

Seguridad

The background of the entire cover is a close-up photograph of a wood surface, showing a prominent vertical grain pattern with varying shades of brown and tan. The texture is smooth but shows natural wood grain variations.

FUNDAMENTALS OF DATABASE SYSTEMS

7TH Edition

ELMASRI • NAVATHE

Seguridad de la información

Tipos de seguridad

La seguridad de la base de datos es un área amplia que aborda muchos problemas, incluidos los siguientes:

Varias cuestiones legales y éticas relacionadas con el derecho a acceder a cierta información; por ejemplo, cierta información puede considerarse privada y no pueden acceder legalmente organizaciones o personas no autorizadas.

Tipos de seguridad

— — —

Cuestiones de política a nivel gubernamental, institucional o corporativo con respecto a
¿Qué tipo de información no debería ponerse a disposición del público?

Por ejemplo, calificaciones crediticias y registros médicos personales.

Tipos de seguridad

La necesidad en algunas organizaciones de identificar múltiples niveles de seguridad y categorizar los datos y los usuarios basándose en estas clasificaciones; por ejemplo, Alto secreto, Secreto, Confidencial y No Clasificado.

La política de seguridad de la organización (con respecto a permitir el acceso a varias clasificaciones de los datos) debe ser aplicada.

Seguridad de la información

— — —

Tanto la **gobernanza** como las cuestiones vinculadas a **privacidad** y **compliance** se apoyan en la capacidad de mantener segura la información. Esto implica:

- ▣ **Confidencialidad:** la información sólo puede ser accedida por los actores autorizados
- ▣ **Integridad:** la precisión y completitud se mantienen evitando que haya modificaciones de la información por actores no autorizados
- ▣ **Disponibilidad:** la información está disponible para aquellos autorizados al momento de necesitarla

Seguridad de la información

— — —

Hay distintas estrategias y técnicas alrededor de la seguridad:

- ▣ **Autenticación:** herramientas de verificación de identidad
- ▣ **Control de acceso:** mecanismos y sistemas para garantizar que los actores autorizados acceden a la información correcta
- ▣ **Encriptación:** mecanismos para oscurecer la información en caso de que caiga en manos incorrectas
- ▣ **Firewalls / sistemas de detección:** herramientas para detectar y prevenir el acceso a redes y sistemas

Medidas de control

Se utilizan distintas medidas de control principales para garantizar la seguridad de los datos en las bases de datos:

— — —

- ▣ Control de inferencia
- ▣ Cifrado de datos (data encryption)
- ▣ Control de acceso

Bases de datos estadísticas

Por ejemplo, una base de datos para estadísticas de población puede proporcionar estadísticas basadas en grupos de edad, niveles de ingresos, tamaño del hogar, niveles de educación y otros criterios.

Los usuarios de bases de datos estadísticas, como estadísticos gubernamentales o empresas de investigación de mercado, pueden acceder a la base de datos para recuperar información estadística sobre una población, pero no para acceder a información confidencial detallada sobre individuos específicos. En estas bases se debe garantizar que no se pueda acceder a la información sobre las personas.

A veces es posible deducir o inferir ciertos hechos relativos a personas a partir de consultas que involucran sólo estadísticas resumidas sobre grupos; en consecuencia, esto tampoco debe permitirse.

Las medidas de control correspondientes se denominan **medidas de control de inferencia**.

Cifrado de datos

Para proteger datos confidenciales (números de tarjetas de crédito, etc.) que se transmite a través de algún tipo de red de comunicación. Los datos se codifican mediante algún algoritmo de codificación. Un usuario no autorizado que acceda a datos codificados tendrá dificultades para descifrarlos.

Los usuarios autorizados reciben algoritmos (o claves) de decodificación o descifrado para descifrar los datos. Hay leyes estatales y federales que prescriben el cifrado para cualquier sistema, que trata sobre información personal legalmente protegida. Por ejemplo, según Ley de Georgia (OCGA 10-1-911)

Seguridad y el DBA

Administrador central de la base de datos, la autoridad para gestionar el sistema.

Las responsabilidades del DBA incluyen otorgar privilegios a los usuarios que necesitan acceder y clasificar usuarios y datos de acuerdo con la política de la organización.

El DBA tiene una cuenta DBA en el DBMS, a veces llamado sistema o cuenta de **superusuario**, que proporciona capacidades que no están disponibles para las cuentas sin privilegios de usuarios.

Los comandos a los que tiene acceso el DBA incluyen comandos para otorgar y revocar privilegios para cuentas individuales, usuarios o grupos de usuarios y para realizar los siguientes tipos de acciones:

1. Creación de cuenta.
2. Concesión de privilegios.
3. Revocación de privilegios.
4. Asignación de nivel de seguridad.

El DBA es responsable de la seguridad general del sistema de base de datos

Control de acceso

Control de acceso discrecional basado en concesión y revocación de privilegios (DAC)

El método típico para hacer cumplir el control de acceso discrecional en un sistema de base de datos se basa en el otorgamiento y revocación de privilegios.

Muchos DBMS relacionales actuales utilizan alguna variación de esta técnica.

La idea principal es incluir declaraciones en el lenguaje de consulta que permitan al DBA y usuarios seleccionados **otorgar y revocar privilegios.**

Control de acceso obligatorio (MAC)

— — —

La técnica DAC de otorgar y revocar privilegios ha sido tradicionalmente el principal mecanismo de seguridad para los sistemas de bases de datos relacionales. Éste es un método de todo o nada: un usuario tiene o no tiene un cierto privilegio.

En muchas aplicaciones, se necesita una política de seguridad adicional que clasifica datos y usuarios según clases de seguridad. Este enfoque, conocido como control de acceso obligatorio (MAC) normalmente se combinará con los mecanismos de control de acceso discrecional.

Es importante tener en cuenta que la mayoría de los RDBMS convencionales proporcionan mecanismos sólo para control de acceso discrecional.

Sin embargo, existe la necesidad de seguridad multinivel.

Control de acceso basado en roles (RBAC)

Su noción básica es que los privilegios y otros permisos están asociados con **roles organizacionales y no con usuarios** individuales. Luego se asignan usuarios individuales a roles apropiados.

Por ejemplo, una empresa puede tener funciones como cuenta de ventas, gerente, agente de compras, gerente de servicio al cliente, etc.

Se pueden asignar varias personas a cada función. Los privilegios de seguridad que son comunes a un rol se otorgan al nombre del rol y a cualquier individuo asignado a este rol automáticamente se le otorgarán esos privilegios.

Control de acceso basado en roles (RBAC)

— — —

- ▣ **Rol:** agrupa funciones y niveles de responsabilidad. Puede ser un administrador, un analista de un área específica, un data entry general, etc. Implican una jerarquía donde cada rol hereda las capacidades de sus ancestros.
- ▣ **Usuario:** cada uno de los miembros de la organización con algún acceso a los datos. Cada usuario podrá tener uno o más **roles**.

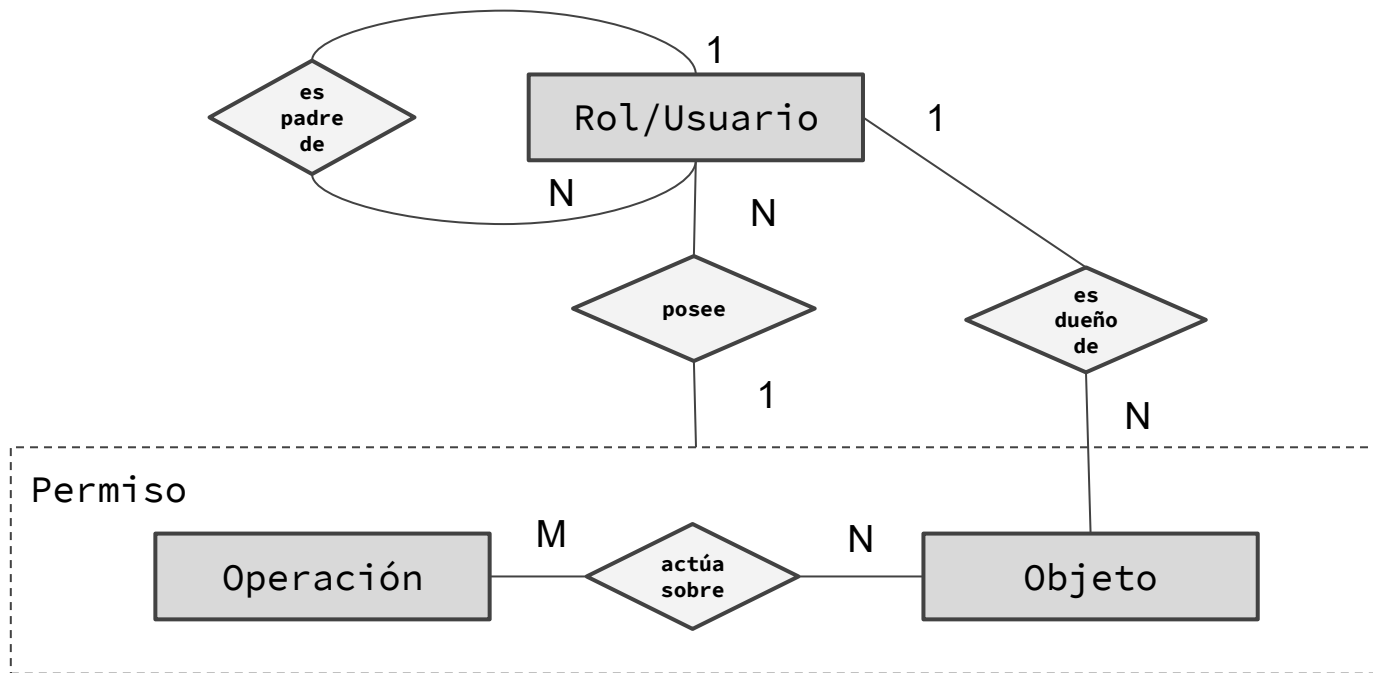
Control de acceso basado en roles (RBAC)

— — —

- ▣ **Objeto:** representa cualquiera de los elementos sobre los que se puede controlar el acceso: tablas, columnas, vistas, esquemas, etc.
- ▣ **Operación:** las distintas acciones que se pueden tomar sobre los objetos: lectura, escritura, borrado, etc.
- ▣ **Permiso:** cada permiso implica **operaciones** sobre **objetos** que están concedidas para un determinado **rol**.

Implementación de RBAC en PostgreSQL

— — —



Implementación de RBAC en PostgreSQL

— — —

Los objetos en PostgreSQL conforman una jerarquía:

- ▣ **Base de datos (database):** puede haber más de una en un mismo motor. Cada una implica un conjunto de **esquemas** y un único sistema de permisos.
- ▣ **Esquema (schema):** separación lógica de objetos dentro de una base de datos. Está compuesta por los objetos atómicos.
- ▣ **Objetos atómicos:** tablas, vistas, funciones, índices, etc.

Implementación de RBAC en PostgreSQL

Las **operaciones** se piensan en términos de **privilegios**:

- **SELECT:** permite leer todas o algunas columnas de una tabla, vista o similar.
- **INSERT:** permite crear registros para una tabla, pudiendo limitar qué columnas pueden escribirse.
- **UPDATE:** permite modificar todas o algunas columnas de una tabla.
- **DELETE:** permite eliminar registros de una tabla o similar.
- **TRUNCATE:** permite hacer TRUNCATE en una tabla
- **CREATE:** permite crear distintos objetos (dependiendo de a qué nivel se otorgue).
- **REFERENCES:** permite crear claves foráneas en todas o algunas columnas de una tabla.
- **TRIGGER:** permite crear triggers en una tabla o similar.
- **ALL PRIVILEGES:** equivale a todos los privilegios posibles sobre un determinado objeto.

Implementación de RBAC en PostgreSQL

— — —

Los **objetos** serán, entre otros:

- **DATABASE:** un conjunto de esquemas.
- **SCHEMA:** un conjunto de objetos atómicos (tablas, vistas).
- **TABLE:** una tabla dentro de un esquema.
- **VIEW:** una vista dentro de un esquema.
- **FUNCTION:** un procedimiento almacenado.

Implementación de RBAC en PostgreSQL

— — —

En PostgreSQL también se maneja el concepto de **owner** de un objeto:

- El **rol** que es **owner** de un objeto tiene automáticamente todos los **privilegios** sobre ese objeto.
- El **rol** que **crea** un objeto es automáticamente **owner**, a menos que especifique otro rol como dueño

RBAC en PostgreSQL - Sintaxis

Para poder administrar permisos, lo primero es crear roles (funcionan como usuarios y como grupos indistintamente):

```
CREATE ROLE nombre_usuario { SUPERUSER | CREATEDB |  
    CREATEROLE | LOGIN | PASSWORD clave |  
    IN ROLE rol_1 , rol_2 , ... | ... };
```

Implementación de RBAC en PostgreSQL

— — —

- **SUPERUSER:** un rol que ignora todas las restricciones de permisos.
- **IN ROLE (IN GROUP):** asigna el nuevo rol como hijo de otro u otros roles existentes (heredando sus permisos).
- **CREATEROLE:** le permite al nuevo rol crear roles.
- **LOGIN:** permite al usuario acceder a una conexión nueva a la base.
- **PASSWORD:** asigna una clave de login al rol.

Ejemplo

- `CREATE ROLE soloLectura;`
- `GRANT CONNECT ON DATABASE unaBaseDeDatos TO soloLectura;`
- `GRANT USAGE ON SCHEMA unEsquemaEnLaBase TO soloLectura;`
- `GRANT SELECT ON TABLE TablaA, TablaB TO soloLectura;`
- `GRANT SELECT ON ALL TABLES IN SCHEMA unEsquemaEnLaBase TO soloLectura;`
- `ALTER DEFAULT PRIVILEGES IN SCHEMA unEsquemaEnLaBase GRANT SELECT ON TABLES TO soloLectura;`

RBAC en PostgreSQL - Sintaxis

— — —

Para crear objetos:

```
CREATE DATABASE nombre_db [ OWNER usuario ];
```

```
CREATE SCHEMA nombre_esquema [ AUTHENTICATION usuario ];
```

```
CREATE TABLE nombre_table ( ... )
```

Donde el owner de los objetos será el rol que los crea a menos que se especifique otro.

RBAC en PostgreSQL - Sintaxis

Modificar (**ALTER**) o eliminar (**DROP**) una tabla sólo puede ser hecho por su owner. Lo mismo ocurre para eliminar una database o un schema.

Pero tanto el owner como un **SUPERUSER** pueden modificar quién es el dueño de un objeto:

```
ALTER TABLE nombre_tabla OWNER TO nombre_rol;
```

```
ALTER DATABASE nombre_db OWNER TO nombre_rol;
```

```
ALTER SCHEMA nombre_schema OWNER TO nombre_rol;
```

RBAC en PostgreSQL - Sintaxis

Para conceder permisos:

```
GRANT { privilegio | ALL [ PRIVILEGES ] }  
      ON { [ TABLE | DATABASE | SCHEMA ] nombre | ALL  
TABLES IN SCHEMA nombre }  
      TO { [ GROUP ] rol [ WITH GRANT OPTION ] }
```

Y para quitarlos:

```
REVOKE privilegio ON objeto FROM rol [ CASCADE |  
RESTRICT ];
```

Implementación de RBAC en PostgreSQL

— — —

- **ALL TABLES IN SCHEMA:** permite dar permisos de tabla para todas las tablas de un schema.
- **WITH GRANT OPTION:** le da permisos de otorgar esos mismos permisos.
- **CASCADE:** revoca también los permisos (de igual característica) concedidos por ese rol a otros roles.

Ataques de seguridad

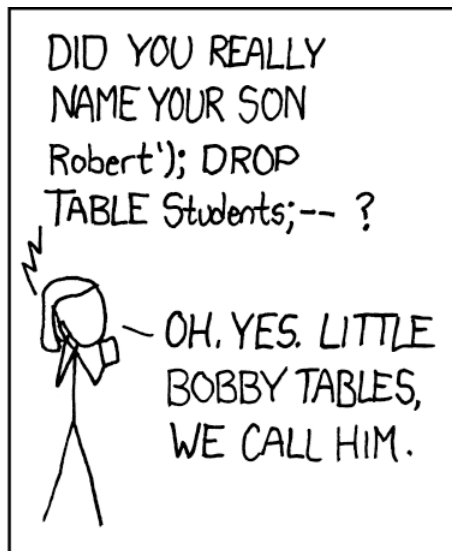
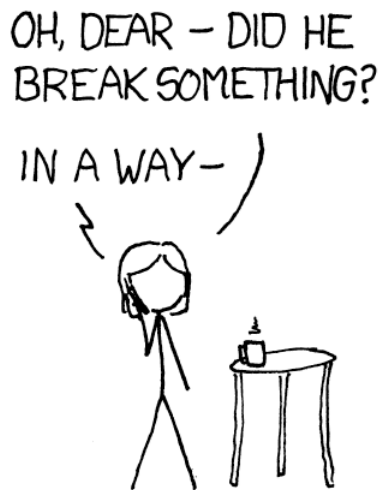
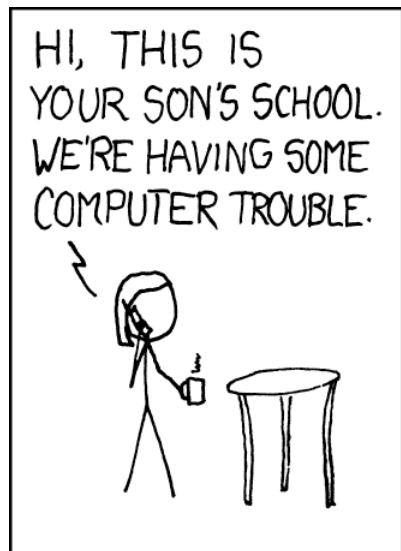
Ataques de seguridad

— — —

- Como todo sistema, los motores de bases de datos relacionales pueden ser víctimas de ataques.
- Por lo general, es un tipo de infraestructura que no va a estar expuesto a redes externas.
- Pero la base de datos es accesible de forma indirecta a través de otros servicios que sí se exponen.

Esto deja lugar a ataques como el de **SQL Injection**

SQL Injection



SQL Injection

— — —

- Son una **falla de seguridad** muy común.
- Explotan el hecho de que muchas veces, los servicios y aplicaciones utilizan **input externo** como parte de sus consultas a la base de datos.
- Los ataques pueden estar dirigidos a:
 - **Obtener** información no autorizada
 - **Alterar** información
 - **Destruir** información (*como en el caso de Bobby Tables*)

SQL Injection

- — — ○ Imaginemos una aplicación que inserta registros en una tabla según valores que recibe de una url. En pseudo código:

```
connection = unServidor\\unaBaseDeDatos
mapear ("/Artist/{ArtistId}/{Name}", (int Id, string Nombre)) {

    string query = "INSERT INTO Artist(ArtistID, name) " +
                   "VALUES ('" + Id + "', '" + Nombre + "')";
}
using = connection{
    SqlCommand command = new SqlCommand(query, connection);
connection.Open();
command.ExecuteNonQuery;
}
```

SQL Injection

--- ○ ¿Y si paso como parámetro lo siguiente?

```
Id = 276,
```

```
Nombre = Paul McCartney') ; DELETE FROM Artist WHERE ('1' = '1
```

SQL Injection

- o Imaginemos un sistema de homebanking que permite consultar operaciones por rango de fecha:

```
def consultar_operaciones(self, fecha_inicio, fecha_fin, cuenta):  
    consulta = ' SELECT * FROM operaciones WHERE cuenta = ' +  
str(cuenta.pk) + ' ' \  
                'AND fecha >= ' + fecha_inicio + ' AND fecha < ' +  
fecha_fin + ';'\  
    resultados = self.db.execute(consulta)  
    return resultados
```

SQL Injection

-
- ¿Cuál es el problema con que `fecha_inicio` y `fecha_fin` sean input de un consumidor?

```
fecha_fin = "2024-01-01'; INSERT INTO operaciones VALUES (...)' "
```

SQL Injection

— — — Para resguardar el sistema es necesario **sanitizar** el input de usuario:

```
string query = "INSERT INTO Artist(ArtistID, name) " +  
               "VALUES(@ArtistID, @name)";  
  
}  
using = connection{  
    SqlCommand command = new SqlCommand(query, connection);  
    command.Parameters.AddWithValue("@ArtistID", ID);  
    command.Parameters.AddWithValue("@name", Nombre);  
    connection.Open();  
    command.ExecuteNonQuery();  
}
```

En lugar de incrustar la entrada del usuario en la declaración, la entrada debe estar vinculada a un parámetro.